CareerTrack – Database Architecture & Technical Documentation

Comprehensive Technical Specification & Schema Guide

1. Database Overview & Core Specifications

The **CareerTrack** Recruitment Management System uses a **document-oriented NoSQL database architecture** powered by **MongoDB** with **Mongoose 8.x ODM** (Object Data Modeling).

Attribute	Specification
Database System	MongoDB 7.x / 8.x
Primary Database Name	careertrack
Connection Protocol	MongoDB Connection URI (mongodb:// or mongodb+srv://)
Local / Test Fallback	mongodb-memory-server (automatic in-memory fallback for instant evaluation)
Data Modeling Layer	Mongoose ODM v8.6.3
Data Integrity Model	Schema-level strict validation, compound unique constraints, and referential integrity

2. Collection Descriptions

The database consists of three core collections, along with embedded subdocuments for notes and resume metadata:

```
careertrack/
├── admins            # Recruiter and administrator authentication & roles
├── jobs              # Job openings, job specifications, requirements & status
└── applications      # Candidate application submissions, résumé links & recruitment
stages
```

Summary of Collections

Collection Name	Mongoose Model	Purpose	Documents Managed
admins	Admin	Stores administrative users, recruiters, hashed credentials, and role-based permissions.	Superadmins, Administrators, Recruiters
jobs	Job	Contains public and internal job postings, department categorization, salaries, criteria, and deadlines.	Active openings, drafts, closed positions
applications	Application	Records candidate submissions per job, embedded résumé file metadata, application status, and recruiter evaluation notes.	Candidate profiles, stage histories

3. Entity-Relationship (ER) Diagram

The following diagram illustrates the entity relationships, key fields, cardinalities, and embedded subdocuments across the CareerTrack database.

```
erDiagram
    ADMIN ||--o{ JOB : "posts / manages (1:N)"
    ADMIN ||--o{ APPLICATION_NOTE : "authors (1:N)"
    JOB ||--o{ APPLICATION : "receives (1:N)"
    APPLICATION ||--|{ RESUME_METADATA : "contains (1:1)"
    APPLICATION ||--o{ APPLICATION_NOTE : "embeds (1:N)"

    ADMIN {
        ObjectId _id PK
        string name
        string email UK "unique, lowercase"
        string password "bcrypt hashed, select:false"
        string role "superadmin | admin | recruiter"
        date lastLogin
        date createdAt
        date updatedAt
    }

    JOB {
        ObjectId _id PK
        string title "indexed, text search"
        string company "indexed, text search"
        string location
```

```
        boolean isRemote
        string employmentType "Full-time | Part-time | Contract | Internship | Temporary"
        string experienceLevel "Entry | Mid | Senior | Lead | Executive"
        string department
        number salaryMin
        number salaryMax
        string salaryCurrency
        boolean isSalaryDisclosed
        string description "text search"
        string_array responsibilities
        string_array qualifications
        string_array skills "indexed, text search"
        string_array benefits
        string applicationInstructions
        string status "Published | Unpublished | Closed | Draft"
        date postingDate
        date applicationDeadline
        number totalApplications "counter cache"
        ObjectId createdBy FK "references admins._id"
        date createdAt
        date updatedAt
    }

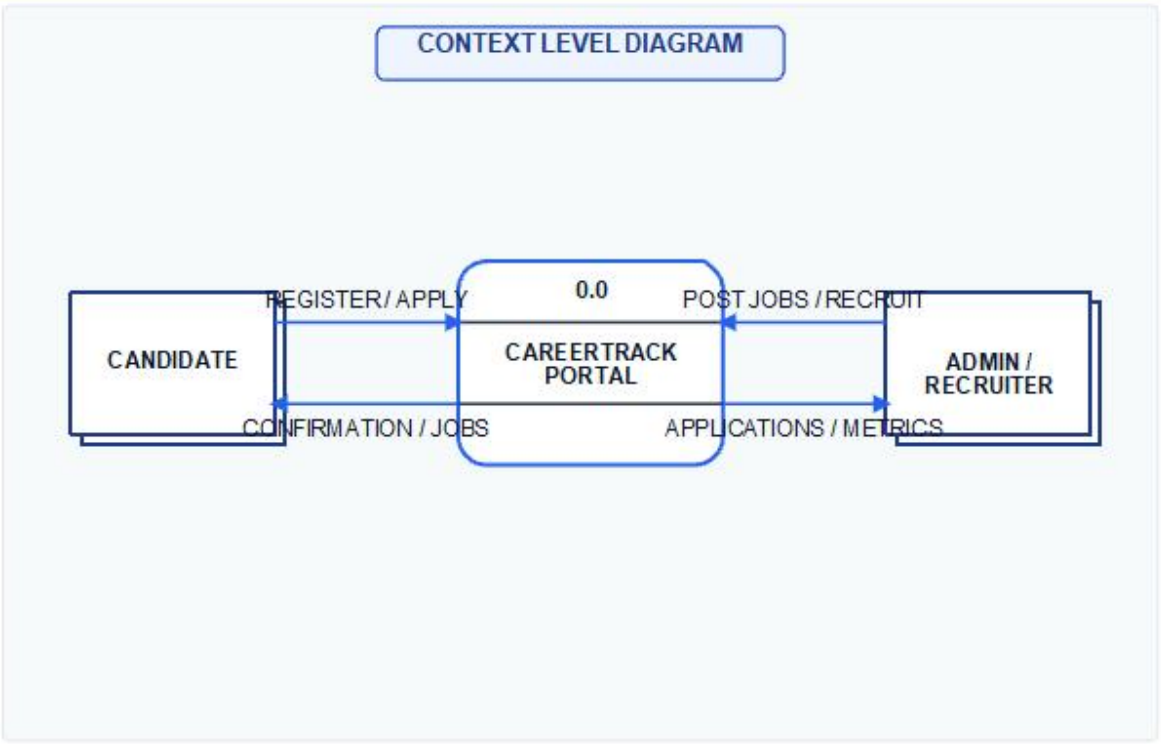
    APPLICATION {
        ObjectId _id PK
        string applicationReferenceId UK "CT-YYYY-XXXXX"
        ObjectId job FK "references jobs._id"
        string candidateName
        string email "lowercase, indexed with job"
        string phone
        string education
        number experienceYears
        string currentRole
        string_array technicalSkills
        string coverLetter
        string status "Submitted | Under Review | Shortlisted | Interview Scheduled |
Selected | Rejected"
        boolean duplicateFlag
        date createdAt
        date updatedAt
    }

    RESUME_METADATA {
        string originalName "e.g. Resume_Jane_Doe.pdf"
        string fileName "unique stored filename"
        string filePath "server storage path"
        string mimeType "application/pdf | application/msword"
        number fileSize "bytes (<= 5MB)"
    }

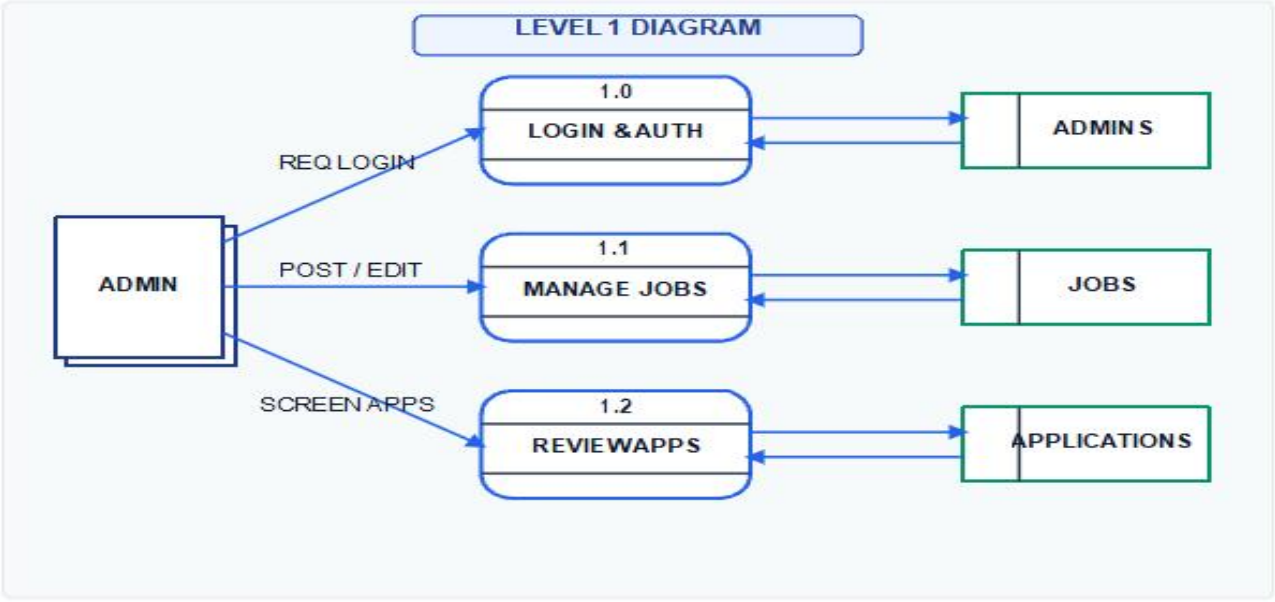
    APPLICATION_NOTE {
```

```
        ObjectId _id PK
        string note "text"
        string authorName "evaluator name"
        ObjectId addedBy FK "references admins._id"
        date createdAt
    }
```

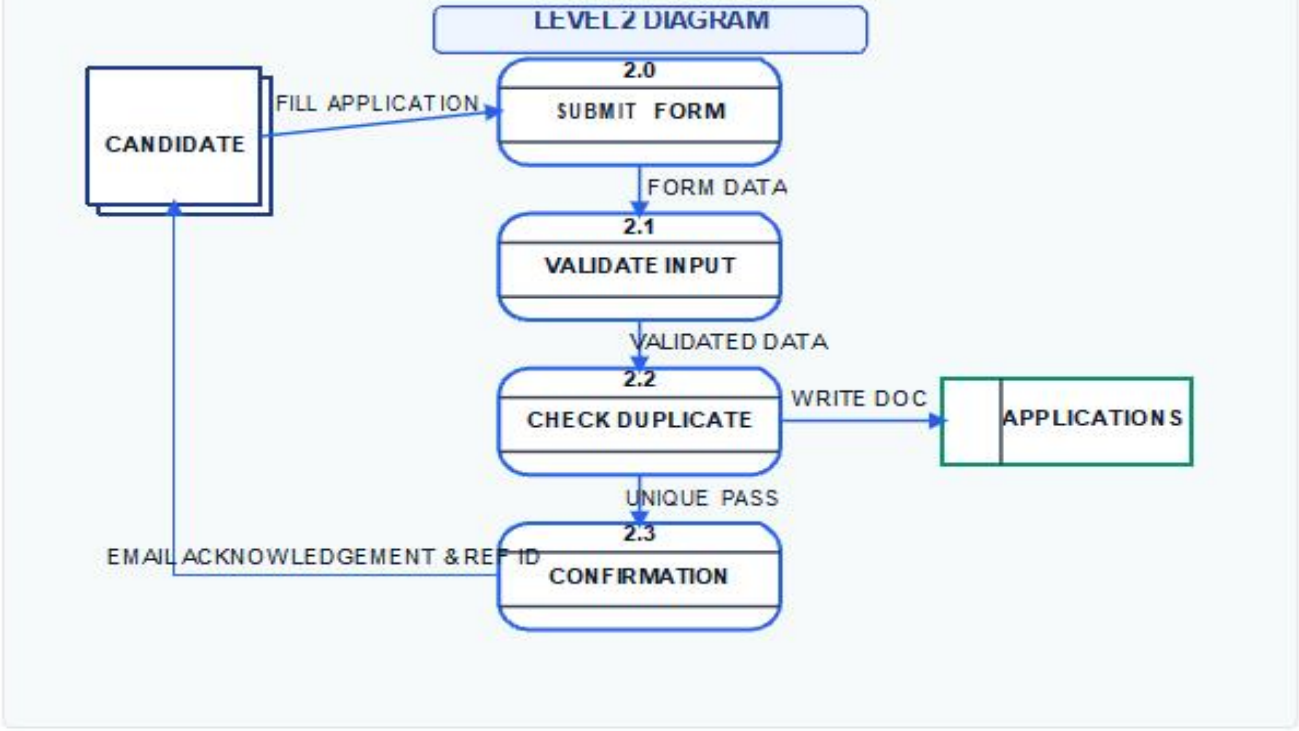
DFD LEVEL 0 (CONTEXT DIAGRAM) OF CAREERTRACK

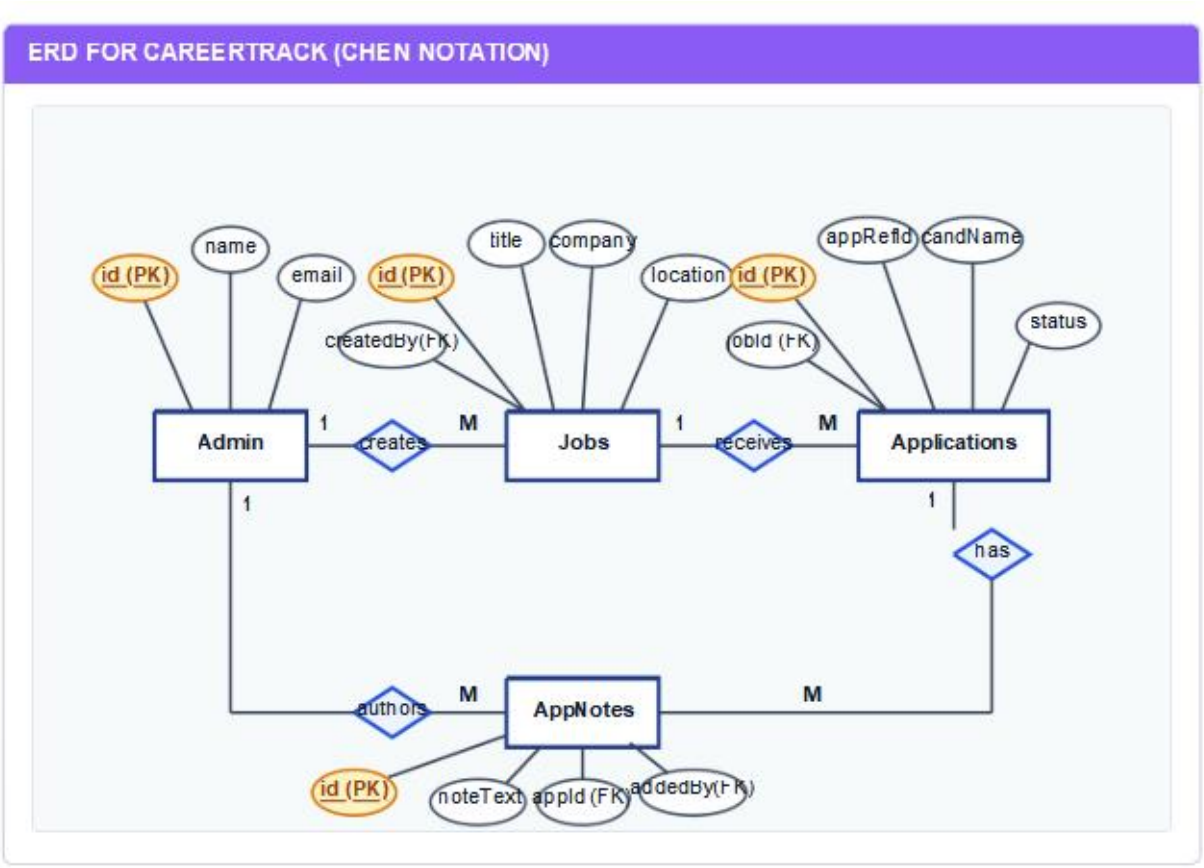


LEVEL 1 DFD OF CAREERTRACK FROM ADMIN SIDE



LEVEL 2 DFD OF APPLICATION SUBMISSION





4. Detailed Schema Explanations

4.1. Job Schema (jobs collection)

The Job schema models all employment vacancies. It includes structured metadata for fine-grained multi-filtering (department, employment type, seniority level, location, remote preference) and search indexing.

```
// backend/src/models/Job.js
const jobSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: [true, 'Job title is required'],
      trim: true,
      maxlength: [120, 'Title cannot exceed 120 characters']
    },
    company: {
      type: String,
```

```
    required: [true, 'Company name is required'],
    trim: true,
    maxlength: [100, 'Company name cannot exceed 100 characters']
  },
  location: {
    type: String,
    required: [true, 'Location is required'],
    trim: true
  },
  isRemote: {
    type: Boolean,
    default: false
  },
  employmentType: {
    type: String,
    required: [true, 'Employment type is required'],
    enum: ['Full-time', 'Part-time', 'Contract', 'Internship', 'Temporary'],
    default: 'Full-time'
  },
  experienceLevel: {
    type: String,
    required: [true, 'Experience level is required'],
    enum: ['Entry-level', 'Mid-level', 'Senior-level', 'Lead', 'Executive'],
    default: 'Mid-level'
  },
  department: {
    type: String,
    trim: true,
    default: 'Engineering'
  },
  salaryMin: {
    type: Number,
    min: [0, 'Minimum salary cannot be negative'],
    default: 0
  },
  salaryMax: {
    type: Number,
    min: [0, 'Maximum salary cannot be negative'],
    default: 0
  },
  salaryCurrency: {
    type: String,
    default: 'USD'
  },
  isSalaryDisclosed: {
    type: Boolean,
    default: true
  },
  description: {
    type: String,
    required: [true, 'Job description is required']
  },
```

```
    responsibilities: {
      type: [String],
      default: []
    },
    qualifications: {
      type: [String],
      default: []
    },
    skills: {
      type: [String],
      default: []
    },
    benefits: {
      type: [String],
      default: []
    },
    applicationInstructions: {
      type: String,
      default: 'Submit your application and résumé through this portal. Our recruitment
team will review submissions on a rolling basis.'
    },
    status: {
      type: String,
      enum: ['Published', 'Unpublished', 'Closed', 'Draft'],
      default: 'Published'
    },
    postingDate: {
      type: Date,
      default: Date.now
    },
    applicationDeadline: {
      type: Date,
      default: null
    },
    totalApplications: {
      type: Number,
      default: 0
    },
    createdBy: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Admin',
      default: null
    }
  },
  { timestamps: true }
);
```

Instance Methods

- **jobSchema.methods.isOpenForApplication()**: Validates that status === 'Published' AND applicationDeadline is either null or in the future.

4.2. Admin Schema (admins collection)

The Admin schema manages credentials and role-based access control (RBAC). It prevents credential exposure by utilizing select: false on password hashes and uses salting through bcryptjs.

```
// backend/src/models/Admin.js
const adminSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Please provide an administrator name'],
      trim: true,
      maxlength: [100, 'Name cannot exceed 100 characters']
    },
    email: {
      type: String,
      required: [true, 'Please provide an email address'],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,})+$/, 'Please provide a valid email
address']
    },
    password: {
      type: String,
      required: [true, 'Please provide a password'],
      minlength: [6, 'Password must be at least 6 characters long'],
      select: false
    },
    role: {
      type: String,
      enum: ['superadmin', 'admin', 'recruiter'],
      default: 'admin'
    },
    lastLogin: {
      type: Date,
      default: null
    }
  },
  { timestamps: true }
);
```

Lifecycle Hooks & Authentication Methods

- **pre('save') Hook:** Automatically generates a 10-round salt and hashes passwords before writing to the database.
- **matchPassword(enteredPassword):** Performs timing-safe comparison between plain password input and stored bcrypt hash.

4.3. Application Schema (applications collection)

The Application schema tracks the candidate's journey from submission to decision. It includes embedded résumé file storage references, recruitment stage tracking, and internal evaluation notes.

```
// backend/src/models/Application.js
const noteSchema = new mongoose.Schema(
  {
    note: { type: String, required: true, trim: true },
    authorName: { type: String, default: 'Admin' },
    addedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'Admin', default: null },
    createdAt: { type: Date, default: Date.now }
  },
  { _id: true }
);

const applicationSchema = new mongoose.Schema(
  {
    applicationReferenceId: {
      type: String,
      unique: true,
      index: true
    },
    job: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Job',
      required: [true, 'Job reference is required'],
      index: true
    },
    candidateName: {
      type: String,
      required: [true, 'Candidate name is required'],
      trim: true,
      maxlength: [100, 'Name cannot exceed 100 characters']
    },
    email: {
      type: String,
      required: [true, 'Candidate email is required'],
      lowercase: true,
      trim: true,
      match: [/^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,})+$/, 'Please enter a valid email
address']
    },
    phone: {
      type: String,
      required: [true, 'Phone number is required'],
      trim: true
    },
    education: {
      type: String,
      required: [true, 'Educational qualification is required'],
```

```
      trim: true
    },
    experienceYears: {
      type: Number,
      required: [true, 'Years of experience is required'],
      min: [0, 'Experience cannot be negative']
    },
    currentRole: {
      type: String,
      trim: true,
      default: ''
    },
    technicalSkills: {
      type: [String],
      default: []
    },
    coverLetter: {
      type: String,
      trim: true,
      default: ''
    },
    resume: {
      originalName: { type: String, required: true },
      fileName: { type: String, required: true },
      filePath: { type: String, required: true },
      mimeType: { type: String, required: true },
      fileSize: { type: Number, required: true }
    },
    status: {
      type: String,
      enum: [
        'Submitted',
        'Under Review',
        'Shortlisted',
        'Interview Scheduled',
        'Selected',
        'Rejected'
      ],
      default: 'Submitted',
      index: true
    },
    notes: {
      type: [noteSchema],
      default: []
    },
    duplicateFlag: {
      type: Boolean,
      default: false
    }
  },
  { timestamps: true }
);
```

Lifecycle Hooks

- **pre('validate') Hook:** Generates a human-friendly tracking identifier (format: CT-YYYY-XXXXX, e.g., CT-2026-A8K2Z) before the document is validated or saved.

5. Relationships Between Jobs and Applications

5.1. Cardinality & Foreign Keys

- **Cardinality:** 1:N (One Job to Many Applications)
- **Foreign Key Reference:** Each document in applications stores a direct reference to the parent job via job: { type: ObjectId, ref: 'Job' }.
- **Bidirectional Querying:**

• Finding all applications for a job:

```
const candidates = await Application.find({ job: jobId }).sort({ createdAt: -1 });
```

• Populating job information from an application:

```
const application = await Application.findById(id).populate('job', 'title company location
department');
```

5.2. Duplicate Submission Prevention

- An application is unique per candidate email + job posting.
- Enforced at the database engine level by a Compound Unique Index:

```
applicationSchema.index({ job: 1, email: 1 }, { unique: true });
```

- **Business Behavior:** A candidate can apply to multiple different jobs, but cannot apply twice to the same job posting. Violations return HTTP 409 Conflict.

5.3. Application Counter Cache

Whenever an application is created, the parent job's totalApplications counter is incremented atomically:

```
await Job.findByIdAndUpdate(jobId, { $inc: { totalApplications: 1 } });
```

5.4. Cascading Deletion

When an administrator deletes a job posting, all associated candidate applications and résumé file paths are cleaned up:

```
await Application.deleteMany({ job: targetJobId });
```

6. Important Fields, Indexes & Validation Rules

6.1. Index Strategy

Indexes are defined to maximize query performance on high-frequency search and filter paths:

Collection	Index Key Pattern	Type	Business Justification
jobs	{ title: 'text', company: 'text', skills: 'text', description: 'text' }	Compound Full-Text	Powers fast public keyword search with relevance scoring.
jobs	{ status: 1, postingDate: -1 }	Compound Index	Optimizes public homepage listings (status: 'Published') sorted by recency.
jobs	{ employmentType: 1, experienceLevel: 1 }	Compound Index	Speeds up multi-filter queries on the candidate portal.
jobs	{ applicationDeadline: 1 }	Single Index	Rapid verification of expired vs active job postings.
admins	{ email: 1 }	Unique Index	Enforces uniqueness of administrative login accounts.
applications	{ applicationReferenceId: 1 }	Unique Index	Fast candidate tracking by reference code (CT-YYYY-XXXXX).
applications	{ job: 1, email: 1 }	Compound Unique	Prevents duplicate applications per email per job posting.
applications	{ status: 1 }	Single Index	Accelerates Admin dashboard pipeline counts & stage filtering.

6.2. Validation Rules & Constraints

Field	Validation Constraint	Error / Behavior
Job.title	required: true, maxlength: 120	Rejects empty or excessively long titles.
Job.employmentType	enum: ['Full-time', 'Part-time', 'Contract', 'Internship', 'Temporary']	Strict enum restriction.
Job.salaryMin / salaryMax	min: 0	Negative salaries are rejected.
Admin.email / Application.email	Regex: /^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,})+$/	Enforces RFC-compliant email formatting.
Admin.password	minlength: 6, select: false	Minimum security threshold; hidden by default.
Application.experienceYears	min: 0	Negative experience values are blocked.
Application.resume.fileSize	<= 5,242,880 bytes (5 MB)	Enforced at Multer middleware layer before DB write.
Application.resume.mimeType	application/pdf, application/msword, application/vnd.openxmlformats...	Only PDF and DOCX formats accepted.

7. Sample Anonymized Records

7.1. Sample Admin Document (admins collection)

```
{
  "_id": { "$oid": "66d01a2b3c4d5e6f7a8b9001" },
  "name": "Sarah Jenkins",
  "email": "admin@careertrack.com",
  "role": "superadmin",
  "lastLogin": { "$date": "2026-11-03T09:42:15.000Z" },
  "createdAt": { "$date": "2026-10-15T08:00:00.000Z" },
  "updatedAt": { "$date": "2026-11-03T09:42:15.000Z" },
  "__v": 0
}
```

(Note: password field is excluded by default via select: false in queries).

7.2. Sample Job Document (jobs collection)


```
{
  "_id": { "$oid": "66d01a2b3c4d5e6f7a8b9010" },
  "title": "Senior Full-Stack Engineer (MERN)",
  "company": "InnovateTech Solutions",
  "location": "San Francisco, CA",
  "isRemote": true,
  "employmentType": "Full-time",
  "experienceLevel": "Senior-level",
  "department": "Engineering",
  "salaryMin": 140000,
  "salaryMax": 185000,
  "salaryCurrency": "USD",
  "isSalaryDisclosed": true,
  "description": "InnovateTech is seeking an experienced Senior Full-Stack Engineer to
architect and develop scalable cloud-native web applications with the MERN stack.",
  "responsibilities": [
    "Architect, build, and maintain scalable full-stack applications with React and
Node.js",
    "Design high-throughput RESTful APIs with robust security standards",
    "Optimize MongoDB schemas, aggregation pipelines, and indexing strategies"
  ],
  "qualifications": [
    "Bachelor's or Master's degree in Computer Science or equivalent",
    "5+ years of production experience in full-stack JavaScript/TypeScript development",
    "Strong proficiency in React.js, Node.js, Express, and MongoDB"
  ],
  "skills": ["React.js", "Node.js", "Express.js", "MongoDB", "TypeScript", "Docker",
"AWS"],
  "benefits": [
    "Competitive equity package and performance bonuses",
    "Comprehensive health, dental, and vision insurance",
    "Remote-first work culture with $1,500 home office stipend"
  ],
  "applicationInstructions": "Upload your updated résumé and a brief cover letter outlining
your experience.",
  "status": "Published",
  "postingDate": { "$date": "2026-10-20T10:00:00.000Z" },
  "applicationDeadline": { "$date": "2026-12-31T23:59:59.000Z" },
  "totalApplications": 24,
  "createdBy": { "$oid": "66d01a2b3c4d5e6f7a8b9001" },
  "createdAt": { "$date": "2026-10-20T10:00:00.000Z" },
  "updatedAt": { "$date": "2026-11-02T14:30:00.000Z" },
  "__v": 0
}
```

7.3. Sample Application Document (applications collection)


```
{
  "_id": { "$oid": "66d01a2b3c4d5e6f7a8b9055" },
  "applicationReferenceId": "CT-2026-K9X2B",
  "job": { "$oid": "66d01a2b3c4d5e6f7a8b9010" },
  "candidateName": "David Chen",
  "email": "david.chen.dev@example.com",
  "phone": "+1 (415) 555-0198",
  "education": "B.S. in Computer Science, UC Berkeley (2020)",
  "experienceYears": 6,
  "currentRole": "Senior Frontend Developer at Horizon Tech",
  "technicalSkills": ["React.js", "Node.js", "TypeScript", "MongoDB", "GraphQL", "Tailwind
CSS"],
  "coverLetter": "Dear Hiring Team,\n\nI am thrilled to apply for the Senior Full-Stack
Engineer opening at InnovateTech. With over 6 years of hands-on experience building
distributed React and Express microservices, I look forward to contributing to your team's
mission.",
  "resume": {
    "originalName": "David_Chen_Resume_2026.pdf",
    "fileName": "resume_1730000000000_david_chen.pdf",
    "filePath": "uploads/resumes/resume_1730000000000_david_chen.pdf",
    "mimeType": "application/pdf",
    "fileSize": 412850
  },
  "status": "Interview Scheduled",
  "duplicateFlag": false,
  "notes": [
    {
      "_id": { "$oid": "66d01a2b3c4d5e6f7a8b9060" },
      "note": "Passed initial technical screening. Strong system design knowledge and React
internals.",
      "authorName": "Sarah Jenkins",
      "addedBy": { "$oid": "66d01a2b3c4d5e6f7a8b9001" },
      "createdAt": { "$date": "2026-10-25T14:15:00.000Z" }
    },
    {
      "_id": { "$oid": "66d01a2b3c4d5e6f7a8b9061" },
      "note": "[Email Sent: Interview Scheduled] Technical Panel round confirmed for
Thursday 2:00 PM PST.",
      "authorName": "Sarah Jenkins",
      "addedBy": { "$oid": "66d01a2b3c4d5e6f7a8b9001" },
      "createdAt": { "$date": "2026-10-27T09:30:00.000Z" }
    }
  ],
  "createdAt": { "$date": "2026-10-21T11:20:00.000Z" },
  "updatedAt": { "$date": "2026-10-27T09:30:00.000Z" },
  "__v": 0
}
```